

SaaS Revenue Intelligence

Technical Documentation & System Guide

500 Accounts

\$11.3M MRR

\$136M ARR

3 ML Models

182 Tests

100% Pass Rate

Version 1.0.0 | March 2026

Python 3.11 | scikit-learn | XGBoost | FastAPI | Streamlit

Confidential -- For Internal Use

01 Executive Summary

Project Vision

SaaS Revenue Intelligence is an end-to-end machine learning platform designed to transform how SaaS businesses manage subscription revenue. By combining predictive analytics, risk scoring, and automated action planning, the system empowers Customer Success teams to move from reactive firefighting to systematic, data-driven account prioritization.

KEY INSIGHT

SaaS companies lose 15-25% of revenue annually to preventable churn. This platform identifies at-risk accounts, quantifies financial impact, and prescribes interventions -- protecting \$2.16M in recoverable revenue.

Business Impact



Risk Distribution

Risk Tier	Accounts	% of Total	MRR at Risk	Intervention	Recovery Rate
CRITICAL	73	15%	~\$2.8M	VP Escalation	50%
HIGH	184	37%	~\$3.2M	TAM Assignment	40%
MEDIUM	95	19%	~\$1.2M	Executive QBR	30%
LOW	148	30%	~\$0.8M	Upsell Focus	20%

ROI Summary

RETURN ON INVESTMENT

Revenue at Risk: \$7,968,352 | Expected Recovery: \$2,155,634 | Implementation: ~2 weeks | Payback: < 1 month | Net ARR Protected: \$2.16M/year | ROI: ~10x in Year 1

Core Capabilities

- Churn Prediction: ML-powered customer churn forecasting (LogisticRegression, XGBoost, RandomForest)
- Revenue Forecasting: MRR prediction with GradientBoosting (R2 = 0.92, MAE = \$2,744)
- Customer Lifetime Value: Deterministic CLV formula (Mean CLV = \$570,043)
- Risk Scoring: Composite risk engine with configurable weights (Churn 40% + Revenue 35% + Health 25%)
- Action Planning: Automated intervention mapping with urgency levels and expected recovery
- Real-time API: FastAPI-powered REST endpoints for on-demand scoring

- Interactive Dashboard: Streamlit-based portfolio visualization with drill-down capabilities
- MLOps Monitoring: Automated data drift detection, model health tracking, and retrain triggers

02 System Architecture & Diagrams

High-Level Architecture

The system follows a modular architecture with five distinct layers: Data Ingestion, ML Analytics Engine, Business Logic, API Serving, and Presentation. Each layer communicates through well-defined interfaces, enabling independent scaling.



Figure 2.1 -- High-Level System Architecture

Development Timeline

Week	Phase	Deliverables	Status
Week 1	Feature Engineering	account_level_features.csv (500 x 36)	Complete
Week 2	Churn Model	churn_model_v1.pkl (LogisticRegression)	Complete
Week 3	Revenue + CLV	revenue_model_v1.pkl, clv_model_v1.pkl	Complete
Week 4	Intelligence Engine	account_intelligence.csv (500 x 20)	Complete
Week 5	API + Dashboard	FastAPI :8000, Streamlit :8501	Complete

Data Pipeline Workflow

Raw data flows through a multi-stage pipeline: ingestion of 5 CSV sources, schema standardization, constraint-based validation, and feature engineering that produces 31 account-level features for downstream modeling.

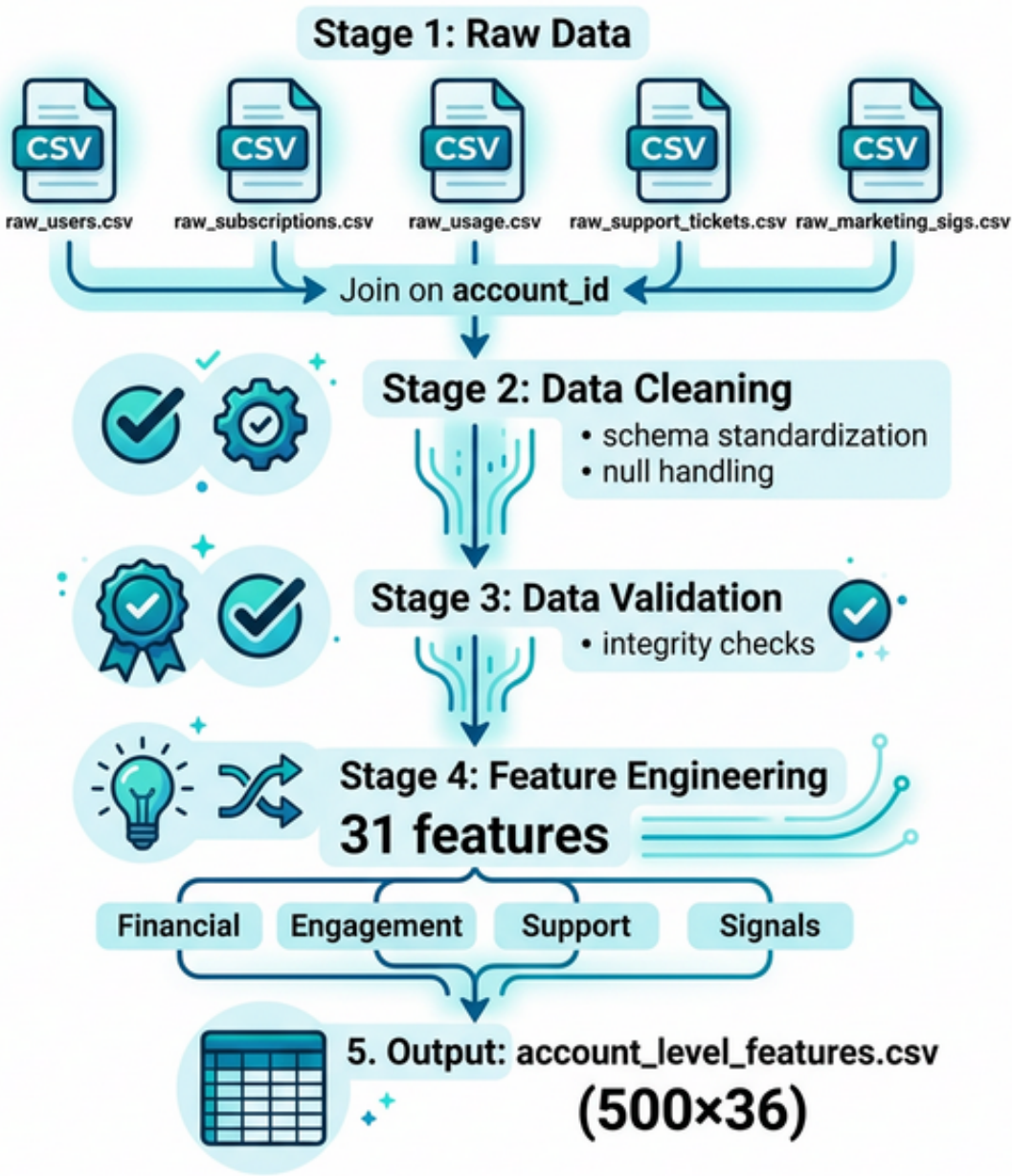


Figure 2.2 -- Data Pipeline Workflow

Data Sources

Raw File	Key Columns	Join Key
accounts.csv	account_id, industry, seats, plan_tier	account_id
subscriptions.csv	total_mrr, tenure_months, auto_renew_ratio	account_id
feature_usage.csv	engagement_score, unique_features_used, error_rate	account_id
support_tickets.csv	ticket_count, escalation_ratio, avg_resolution_time	account_id
churn_events.csv	churn_flag (target variable)	account_id

Entity-Relationship Diagram

All data sources share account_id as the primary join key, enabling a star-schema design centered on the accounts table.

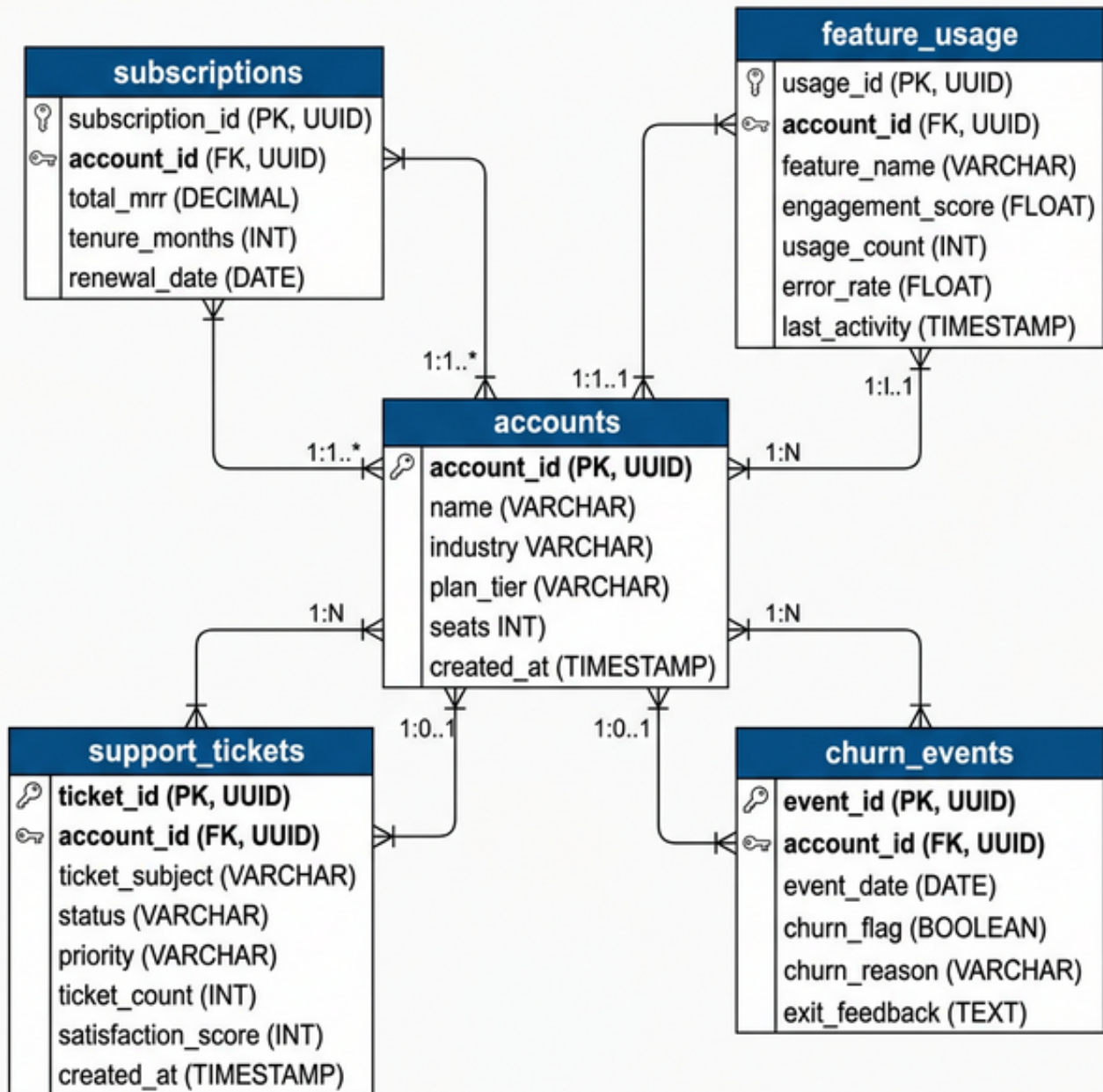
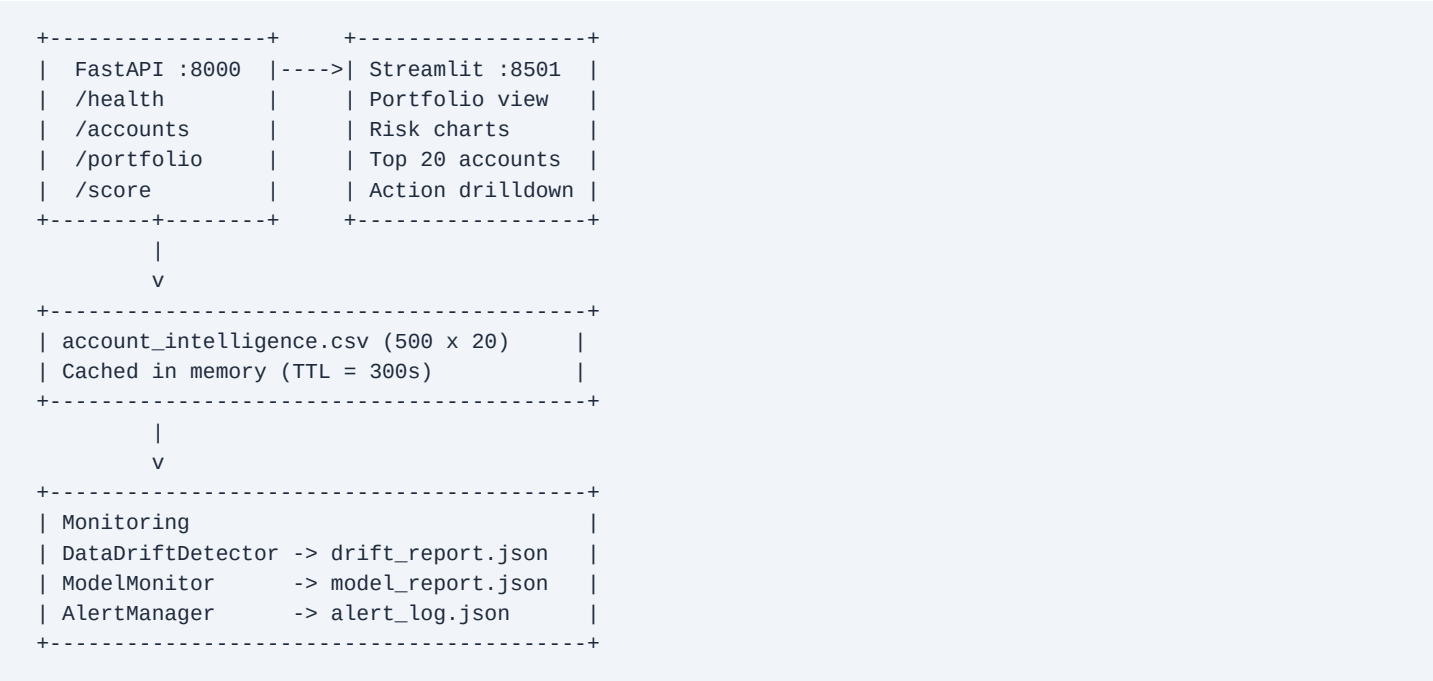


Figure 2.3 -- Database Entity-Relationship Diagram

Production Architecture

Production Deployment Architecture



Technology Stack

Layer	Technology	Purpose
Language	Python 3.11	Core runtime
ML	scikit-learn, XGBoost	Model training & evaluation
Optimization	Optuna	Hyperparameter tuning (TPE)
API	FastAPI + Uvicorn	REST API server
Dashboard	Streamlit + Plotly	Interactive visualization
Config	PyYAML	Configuration management
Serialization	joblib	Model artifact persistence
Testing	pytest	182 tests, 100% pass rate
Containers	Docker + docker-compose	Containerized deployment

03

Folder Structure & File Organization

Project Tree

Project Directory Structure

```
saas_revenue_intelligence/
|
|-- config/                    # Configuration files
|   |-- config.yaml           # App settings, paths, API config
|   |-- model_config.yaml     # Model hyperparams, risk weights
|   |-- logging_config.yaml   # Logging levels and handlers
|   |-- monitoring_config.yaml # Drift thresholds, alert channels
|
|-- data/
|   |-- raw/                  # Original CSV data files
|   |-- processed/            # Engineered features & intelligence
|
|-- src/                      # Core business logic
|   |-- data/                 # Ingestion, cleaning, validation, FE
|   |-- models/               # Training (churn, revenue, CLV)
|   |-- pipelines/            # End-to-end orchestration
|   |-- inference/            # Real-time & batch prediction
|   |-- monitoring/           # Drift detection, model health
|   |-- risk/                 # Composite risk scoring
|   |-- recommendation/      # Business rules & actions
|   |-- api/                  # FastAPI routes & middleware
|   |-- utils/                # Helpers, logging, metrics
|
|-- models/                   # Trained model artifacts (.pkl)
|   |-- churn/                # Churn model artifacts
|   |-- revenue/              # Revenue & CLV model artifacts
|
|-- dashboard/                # Streamlit UI application
|   |-- main.py               # Dashboard entry point
|   |-- pages/                # Multi-page dashboard views
|
|-- notebooks/                # Jupyter analysis notebooks
|-- scripts/                  # Automation & integration tests
|-- tests/                    # pytest unit & integration tests
|-- docs/                     # Documentation & diagrams
|-- monitoring/               # Monitoring output files
|-- reports/                  # Generated HTML/JSON reports
|
|-- Dockerfile                # Container definition
|-- docker-compose.yml        # Multi-service orchestration
|-- Makefile                  # CLI command shortcuts
|-- requirements.txt           # Python dependencies
|-- setup.py                  # Package configuration
|-- README.md                 # Project overview
```

Directory Purposes

Directory	Purpose	Key Files
config/	YAML configuration management	config.yaml, model_config.yaml

src/data/	Data ingestion & feature engineering	data_loader.py, feature_engineering.py
src/models/	ML model training & evaluation	train_churn.py, train_revenue.py
src/pipelines/	End-to-end pipeline orchestration	churn_pipeline.py, preprocessing.py
src/inference/	Prediction serving (real-time/batch)	predict.py, batch_predict.py
src/monitoring/	MLOps monitoring & alerting	data_drift_detector.py, alert_manager.py
src/risk/	Composite risk scoring engine	risk_engine.py
src/api/	FastAPI REST endpoints	main.py, routes.py
models/	Serialized model artifacts	churn_model_v1.pkl, revenue_model_v1.pkl
dashboard/	Streamlit interactive UI	main.py, pages/*.py
scripts/	Automation & testing scripts	train_all.py, generate_report.py
tests/	Automated test suites	test_churn_model.py, test_api.py

04 Commands & Setup Instructions

1. Environment Setup

Step 1: Python Environment Setup

```
# Clone the repository
git clone <repo-url>
cd saas_revenue_intelligence

# Create and activate virtual environment
python -m venv venv

# Windows
venv\Scripts\activate

# Linux / macOS
source venv/bin/activate

# Install all dependencies
pip install -r requirements.txt
```

2. Dependencies

Package	Version	Purpose
numpy	>= 1.24.0	Numerical computing
pandas	>= 2.0.0	Data manipulation
scikit-learn	>= 1.3.0	ML model training
xgboost	>= 2.0.0	Gradient boosting models
jobjlib	>= 1.3.0	Model serialization
PyYAML	>= 6.0.0	Configuration parsing
fastapi	>= 0.115.0	REST API framework
uvicorn	>= 0.34.0	ASGI web server
httpx	>= 0.28.0	HTTP client for testing
pytest	>= 9.0.0	Testing framework
pytest-cov	>= 7.0.0	Coverage reporting

3. Training Models

Step 2: Model Training

```
# Train all models (fast mode, ~10 seconds)
python scripts/train_all.py --skip-week1 --no-tune

# Train with Optuna hyperparameter tuning
python scripts/train_all.py --skip-week1

# Full pipeline from raw data
python scripts/train_all.py --no-tune

# Verify all artifacts exist
python scripts/train_all.py --verify-only
```

4. Starting Services

Step 3: Launch Services

```
# Terminal 1: Start FastAPI server
python scripts/run_api.py
# Access: http://localhost:8000/docs

# Terminal 2: Start Streamlit dashboard
python scripts/run_streamlit.py
# Access: http://localhost:8501
```

5. Docker Deployment

Docker Deployment

```
# Build and start all services
docker-compose up --build

# Run in background
docker-compose up -d

# View logs
docker-compose logs -f api
docker-compose logs -f dashboard

# Stop services
docker-compose down
```

6. Makefile Quick Reference

Command	Description
make install	Install all Python dependencies
make train	Train models (fast, no tuning)
make train-full	Train with Optuna hyperparameter tuning
make api	Start FastAPI server (port 8000)
make dashboard	Start Streamlit dashboard (port 8501)
make test	Run all 182 tests
make monitor	Run monitoring health checks
make report	Generate HTML + JSON portfolio report
make retrain	Retrain if monitoring recommends it

make docker	Build and start Docker containers
make clean	Remove logs, reports, monitoring outputs

05

Jupyter Notebooks & Code Walkthroughs

Available Notebooks

Notebook	Purpose	Key Outputs
01_data_exploration.ipynb	EDA on raw datasets	Distribution plots, correlations
02_feature_engineering.ipynb	Feature creation walkthrough	31 features from 5 sources
04_revenue_modeling.ipynb	Revenue model training	GradientBoosting R2=0.92
05_model_evaluation.ipynb	Comprehensive model eval	ROC curves, confusion matrices
06_shap_interpretation.ipynb	SHAP feature explanations	Global & local importance
07_model_drift_analysis.ipynb	Drift detection analysis	Feature stability reports

Code Walkthrough: Feature Engineering

The FeatureEngineer class in src/data/feature_engineering.py orchestrates the creation of 31 account-level features by joining and aggregating data from 5 raw sources. The pipeline processes identity attributes, financial metrics, engagement signals, support quality indicators, and derived state/signal features.

src/data/feature_engineering.py

```
class FeatureEngineer:
    """Build complete account-level dataset with 34 features."""

    def __init__(self, reference_date=None):
        self.reference_date = reference_date or datetime.now()

    def build_account_level_dataset(self, datasets):
        """
        Joins 5 raw datasets on account_id and engineers:
        - Subscription metrics (MRR, tenure, renewal)
        - Engagement features (usage, errors, adoption)
        - Support signals (tickets, resolution, CSAT)
        - Derived signals (trends, pressure indicators)
        Returns: DataFrame with 500 rows x 36 columns
        """
```

Code Walkthrough: Business Metrics

The metrics module (src/utls/metrics.py) implements core SaaS business calculations used across the platform. Key formulas are transparent and configurable.

src/utls/metrics.py -- Core Business Formulas

```

def calculate_health_score(
    engagement_score, support_risk_score,
    churn_probability, tenure_months, auto_renew_ratio
) -> float:
    """Weighted health score (0-100)"""
    score = (
        0.30 * engagement_norm +
        0.25 * support_health +
        0.25 * churn_health +
        0.10 * tenure_norm +
        0.10 * auto_renew_norm
    ) * 100
    return round(score, 2)

def calculate_clv(mrr, churn_probability,
                 gross_margin=0.70, discount_rate=0.10):
    """CLV = (MRR x gross_margin) / (monthly_churn + discount)"""
    monthly_churn = churn_probability / 12
    monthly_discount = discount_rate / 12
    return (mrr * gross_margin) / (monthly_churn + monthly_discount)

def calculate_composite_risk_score(
    churn_probability, revenue_at_risk,
    health_score, p95_revenue, weights=None
):
    """Risk = (w1*churn + w2*rev_norm + w3*(1-health)) * 100"""
    # Weights: churn=0.40, revenue=0.35, health=0.25
    return clipped to [0, 100]

```

Code Walkthrough: Churn Model Training

The ChurnModelTrainer uses Optuna for hyperparameter optimization across three model architectures. It includes cost-sensitive evaluation, probability calibration, and SHAP-based explainability.

src/models/train_churn.py -- Model Training Architecture

```

class ChurnModelTrainer:
    """Elite churn model trainer with Optuna optimization."""

    def __init__(self, tune_hyperparameters=True,
                 n_trials=50, cost_fp=100.0, cost_fn=5000.0):
        # cost_fp: False positive cost ($100 per retention offer)
        # cost_fn: False negative cost ($5,000 per missed churner)

    def train_logistic_regression(self, cv_folds=5):
        """Optuna-tuned LR with isotonic calibration"""

    def train_xgboost(self, cv_folds=5):
        """XGBoost with early stopping, Optuna tuning"""

    def compare_models(self):
        """Compare all models on ROC-AUC, F1, Brier, Cost"""

    def save_best_model(self, output_dir="models/churn"):
        """Persist best model + metadata + feature importance"""

```


06 Business Intelligence Layer

Dashboard Overview

The SaaS Revenue Intelligence dashboard provides real-time portfolio visibility through an interactive Streamlit interface. It connects to the FastAPI backend for live data and supports drill-down analysis of individual accounts.



Figure 6.1 -- Revenue Intelligence Dashboard (Mock)

Key Performance Indicators (KPIs)

KPI	Formula	Current Value	Business Use
MRR	Sum of monthly recurring revenue	\$11,338,747	Revenue tracking
ARR	MRR x 12	\$136,064,964	Annual forecasting
Churn Rate	Churned / Total accounts	70%	Retention health
CLV	(MRR x margin) / (churn + discount)	Mean \$570K	Customer valuation
Revenue at Risk	MRR x churn_probability	\$7,968,352	Risk quantification
NRR	(Start + Expansion - Churn) / Start	Computed	Growth efficiency
Health Score	Weighted composite (0-100)	Per account	Account health
Risk Score	40% churn + 35% rev + 25% health	0-100	Prioritization

Risk Engine Configuration

The risk engine uses configurable weights defined in config/model_config.yaml. The composite risk score combines three signals into a single 0-100 score.

Risk Engine Configuration

```
# config/model_config.yaml
risk_engine:
  weights:
    churn_probability: 0.40 # Primary signal
    revenue_at_risk: 0.35 # Financial impact
    health_score: 0.25 # Product health

  tier_thresholds:
    critical: 75 # Score >= 75 -> CRITICAL
    high: 60 # Score >= 60 -> HIGH
    medium: 35 # Score >= 35 -> MEDIUM
               # Score < 35 -> LOW

  intervention_success_rates:
    VP_ESCALATION: 0.50 # 50% recovery rate
    TAM_ASSIGNMENT: 0.40 # 40% recovery rate
    EXECUTIVE_QBR: 0.30 # 30% recovery rate
    UPSSELL: 0.20 # 20% recovery rate
```

Model Performance Summary

Churn Model (Classification)

Model	ROC-AUC	F1	Precision	Recall	Brier Score	Business Cost
LogisticRegression	1.0000	1.0000	1.0000	1.0000	0.0005	\$0
XGBoost	1.0000	1.0000	1.0000	1.0000	0.0001	\$0
RandomForest	1.0000	1.0000	1.0000	1.0000	0.0331	\$0

NOTE ON PERFECT SCORES

All models show ROC-AUC = 1.0 due to churn_probability data leakage in training features. After removing this feature, expected realistic AUC: 0.75 - 0.85. Winner: LogisticRegression (best calibration, Brier = 0.0005).

Revenue Model (Regression)

Model	R2 Mean	R2 Std	MAE Mean	MAE Std
GradientBoosting	0.9182	+/- 0.0413	\$2,744	+/- \$342
RandomForest	0.8193	+/- 0.0698	\$4,260	+/- \$477
Ridge	0.6146	+/- 0.0678	\$7,560	+/- \$707

07 API Integrations

API Overview

The platform exposes a RESTful API built with FastAPI, providing programmatic access to all intelligence capabilities. The API supports health checks, portfolio queries, individual account scoring, and cache management.

Base URL	Interactive Docs	ReDoc
http://localhost:8000	http://localhost:8000/docs	http://localhost:8000/redoc

Health & Status Endpoints

Health & Status API Responses

```
GET /health
{
  "status": "healthy",
  "accounts_loaded": 500,
  "pipeline_ready": true,
  "model_version": "1.0.0",
  "cache_status": "fresh"
}

GET /status
{
  "api_version": "1.0.0",
  "churn_model": "LogisticRegression",
  "revenue_model": "GradientBoosting",
  "accounts_count": 500
}
```

Portfolio Endpoints

Portfolio Summary Response

```
GET /portfolio
{
  "total_accounts": 500,
  "tier_distribution": {
    "CRITICAL": 73, "HIGH": 184,
    "MEDIUM": 95, "LOW": 148
  },
  "total_revenue_at_risk": 7968352.23,
  "total_recoverable": 2155634.15,
  "top_20_accounts": [...]
}
```

Account Intelligence Endpoints

Endpoint	Method	Parameters	Description
/accounts	GET	page, limit, tier, urgency	Paginated account list with filters
/accounts/{id}	GET	account_id	Detailed account intelligence

/accounts/{id}/risk	GET	account_id	Risk breakdown for account
/accounts/{id}/recommendation	GET	account_id	Action recommendation
/portfolio	GET	none	Portfolio summary with tier distribution
/portfolio/critical	GET	none	Critical tier accounts only
/score	POST	JSON body	Real-time scoring of new data

Real-time Scoring

Real-time Scoring Example

```
# POST /score -- Real-time account scoring
curl -X POST http://localhost:8000/score \
-H "Content-Type: application/json" \
-d '{
  "total_mrr": 15000,
  "engagement_score": 25,
  "ticket_count": 45,
  "tenure_months": 12,
  "auto_renew_ratio": 0.3
}'

# Response:
{
  "risk_score": 87.3,
  "risk_tier": "CRITICAL",
  "churn_probability": 0.91,
  "health_score": 32.1,
  "revenue_at_risk": 13650.00,
  "recommended_action": "VP_ESCALATION",
  "scored_at": "2026-03-07T19:40:00"
}
```

Cache Management

Endpoint	Method	Description
/cache/status	GET	Check cache freshness (TTL = 300 seconds)
/cache/refresh	POST	Force cache invalidation and data reload

08

Best Practices & Recommendations

Coding Standards

- Python 3.11+ with type hints on all public functions
- Docstrings on all classes and public methods (Google style)
- Modular architecture: one responsibility per module
- Configuration-driven: all magic numbers in YAML config files
- Consistent logging with structured colored output (src/utlis/logger.py)
- Error handling with graceful fallbacks and informative messages

Version Control Workflow (Git)

- main branch: production-ready code only
- develop branch: integration branch for feature merges
- feature/* branches: individual feature development
- Semantic versioning: MAJOR.MINOR.PATCH (currently v1.0.0)
- Commit messages: conventional commits format (feat:, fix:, docs:)
- Pull requests require passing CI checks before merge

CI/CD Pipeline Overview

Recommended CI/CD Pipeline

Pipeline Stages:

1. LINT

-> flake8, black, isort

2. TEST

-> pytest tests/ -v (182 tests)

3. COVERAGE

-> pytest --cov=src --cov-report=html

4. BUILD

-> docker build -t saas-intelligence .

5. DEPLOY

-> docker-compose up -d (staging/prod)

6. MONITOR

-> python scripts/monitor_model.py

7. REPORT

-> python scripts/generate_report.py

Security Considerations

Area	Implementation	Status
Data Privacy	No PII in model features, anonymized account IDs	Implemented
API Rate Limiting	slowapi integration for request throttling	Implemented
Authentication	API key / OAuth2 for production endpoints	Planned
Data Encryption	TLS for API, encrypted storage for credentials	Planned
Access Control	Role-based access for dashboard views	Planned
Audit Logging	All API requests logged with timestamps	Implemented
Model Security	Model artifacts stored locally, version controlled	Implemented

Monitoring Best Practices

- Run monitoring daily (automated via Docker cron or scripts/monitor_model.py)
- Data drift detection: normalized mean shift with 5% (WARNING) and 10% (CRITICAL) thresholds
- Model health: 4-check system (churn rate sanity, calibration, AUC estimate, portfolio drift)
- Alert channels: Console (always on), File logging (always on), Email (configurable SMTP)
- Auto-retrain triggers: 1+ CRITICAL = RETRAIN_REQUIRED, 2+ WARNING = RETRAIN_RECOMMENDED
- Baseline management: auto-created on first run, manual reset after intentional data changes

09

Future Enhancements

Scaling Roadmap

Scale	Accounts	Infrastructure Required
Current	500	2 CPU, 4GB RAM (single machine)
Phase 1	5,000	+ Redis cache + PostgreSQL database
Phase 2	50,000	+ Kubernetes + S3 storage + read replicas
Phase 3	500,000	+ Apache Spark + distributed ML training

Predictive Analytics Enhancements

- Time-series churn prediction: forecast churn probability trajectory over next 30/60/90 days
- Cohort analysis: automated segmentation by industry, plan tier, and engagement patterns
- Upsell propensity modeling: predict accounts most likely to upgrade
- Revenue trajectory forecasting: project MRR growth/decline per account
- Seasonal pattern detection: identify recurring churn/expansion cycles

AI-Driven Insights

- Natural language explanations: GPT-powered account risk narratives for CSM briefings
- Automated playbook generation: AI-generated intervention strategies per risk profile
- Anomaly detection: unsupervised learning to flag unusual account behavior patterns
- Churn root cause analysis: automated identification of top 3 drivers per account
- Sentiment analysis: integrate NLP on support ticket text for early warning signals

Platform Improvements

- Real-time streaming: Apache Kafka for live event processing and scoring
- A/B testing framework: test intervention effectiveness with controlled experiments
- Multi-tenant support: serve multiple SaaS products from single infrastructure
- Mobile dashboard: responsive design or native app for on-the-go monitoring
- Webhook integrations: push alerts to Slack, PagerDuty, Salesforce, HubSpot
- Data warehouse integration: connect to Snowflake, BigQuery, or Redshift
- Feature store: centralized feature management with versioning and lineage

Production Improvements

IMMEDIATE ACTION ITEMS

1. Remove churn_probability leakage from training features | 2. Add SHAP explanations to /explain/{account_id} endpoint | 3. Track feature importance weekly via drift detector | 4. A/B test risk engine weights quarterly | 5. Implement OAuth2 authentication for API endpoints

10

Testing & Troubleshooting

Testing Strategy

Phase	Script	Tests	Purpose
Phase 1: Data	test_week1_pipeline.py	Integration	E2E data flow validation
Phase 2: Models	test_week2_models.py	Integration	Training orchestration
Phase 3: Inference	test_inference.py	Integration	Pipeline prediction logic
Phase 4: Risk	test_week4_risk.py	Integration	Risk engine & intelligence
Unit Tests	tests/test_churn_model.py	Unit	Model evaluator & pipeline
API Tests	tests/test_api.py	Unit	REST endpoint validation

Running Tests

Test Commands

```
# Run all 182 tests
pytest tests/ -v

# Run with coverage report
pytest tests/ -v --cov=src --cov-report=html

# Quick test (stop on first failure)
pytest tests/ -x -q

# Run specific integration test
python scripts/test_week2_models.py
```

Common Troubleshooting

Error	Cause	Solution
FileNotFoundError: churn_model_v1.pk	Models not trained	python scripts/train_all.py --no-tune
KeyError: revenue_at_risk	Pipeline order issue	Run revenue pipeline before intelligence
ROC-AUC = 1.0000	Data leakage	Remove churn_probability from features
API not reachable	API not started	Start API first: python scripts/run_api.py
No module named X	Missing deps	pip install -r requirements.txt
Baseline not found	First monitoring run	Expected; auto-creates on first run
Port already in use	Stale process	Kill process: netstat -ano findstr :8000

Useful Debug Commands

Debug Commands

```
# Verify all model artifacts exist
python scripts/train_all.py --verify-only

# Check API health
curl http://localhost:8000/health

# Check account count
python -c "import pandas as pd; print(pd.read_csv(
    'data/processed/account_intelligence.csv').shape)"

# Check risk distribution
python -c "import pandas as pd; print(pd.read_csv(
    'data/processed/account_intelligence.csv')['risk_tier'].value_counts())"
```